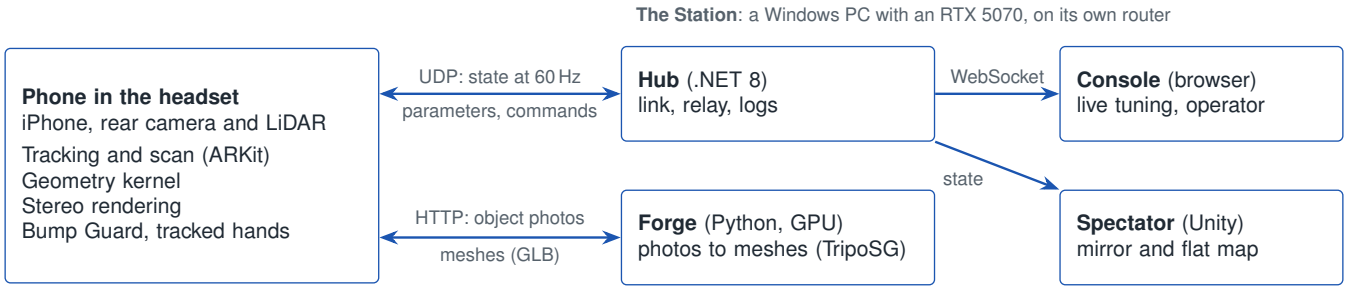
ATLAS: technical guide

Every feature and how it is built. Enough to explain the work and answer questions.

4 October 2026. Taken from the code, the contract pages (`docs/contracts/`) and the Master Plan. Where this guide and the code differ, the code is right.

1 ATLAS on one page

ATLAS is our entry for Huawei Custom Challenge 1, “Beyond Euclid”. A phone in a lens headset scans your room; the room becomes the first tile of other universes, and you walk through them on your own feet.



- **Phone:** iPhone 15 Pro Max or 14 Pro Max. Unity 6 (6000.3.25f1), URP, AR Foundation 6.5, C#, with small Swift plugins.
- **The app is complete without the Station.** With the Hub stopped, only the link indicator changes.
- **Two layers, never confused.** The *world layer* is everything the walker sees: generated, and flat, curved, glued or four-dimensional. The *physical layer* is the scan: where the real floor, walls and furniture are. It is never drawn, and only Bump Guard reads it. In a flat space the two coincide; in a curved space they cannot, which is why safety is its own system (section 6).

The spaces

One geometry kernel, one shader and one host scene serve all of them, and every space is generated from the scanned room.

	Space	Geometry	What you feel
S0	Home	flat	Your room, rebuilt 1:1. Every journey starts and ends here.
S1	Glued	flat, wrapped: torus, Klein bottle, 3-torus	Look ahead and see your own back. Walk out one side, come in the other.
S2	Curved Walk	hyperbolic floor, {4,5} tiling	Five rooms meet at every corner. A square walked on the floor is not closed in the world.
S3	Aligned Cells	the same tiling, from flat pieces	Walk once round a corner post and you are in a fifth room.
S4	Small Planet	spherical floor, six tiles	Three right-angle turns make a closed triangle. The world is finite.
S5	Impossible Rooms	flat rooms joined by portals	The building is bigger than the room.
S6	Slicer	four dimensions, as 3D slices	A solid changes shape as it moves in a direction you cannot point to.
S7	Network Atlas	a network in the hyperbolic plane	A network laid out in hyperbolic space; a packet finds its way by always stepping closer.

2 Scanning the room

The walker sweeps the room with the phone in hand. The result is a `RoomModel`: the one description of the real room that every space, the Forge and Bump Guard use.

Sensor data	From	Used for
Camera and IMU	ARKit world tracking, 60 Hz	Head pose every frame; key frames for the object photos.
LiDAR mesh with surface classes, planes	`ARMeshManager`, `ARPlaneManager`	Floor, obstacles and wall directions. On only during the scan, to save heat.
LiDAR depth image	`AROcclusionManager`	Stays on after the scan, for Bump Guard and the hands.
RoomPlan furniture boxes	`ARBoundingBoxManager`	Each object's box and category (chair, table, bed, . . .).

1. **Grid.** The mesh is dropped onto a 5 cm floor grid. A cell is *floor* if floor-class triangles lie within 0.08 m of floor height, and *obstacle* if anything stands 0.10 to 2.0 m above it. A cell nobody saw is never free.
2. **Distance field.** For every cell, the distance to the nearest obstacle or wall; it reads 0.05 m short, on the safe side. Walkable floor is where it exceeds 0.25 m.
3. **Free rectangle.** The largest rectangle of walkable floor along the main wall direction, $W_f \times D_f$. It sets the lap square s and so the curvature radius L (section 3). Under 1.5 m the app uses standing mode.
4. **Walls and objects.** Walls come from the outline's edges. Objects are RoomPlan's boxes merged with the obstacle blobs of the grid, at most 64.
5. **Confirm and anchor.** The walker confirms the outline, and an `ARAnchor` is placed at the rectangle's centre.
6. **Photos.** Up to three crops per object go to the Forge (section 8).

- **Registration.** Everything in the model is stored relative to the anchor. Every frame the pose pipeline moves the XR origin so that the anchor is the world origin, so when ARKit corrects its own drift the room moves with it.
- **Home (S0)** is the model drawn 1:1: floor, walls and door openings rebuilt on a 0.25 m grid, and one mesh per object.
- The solve (`FloorScanCore`, then `RoomModelBuilder`) runs on a worker thread, so the picture never stalls.

Status. The scan has run to the end on a phone (4 October). Accuracy is not measured yet; the targets are a free rectangle within 0.10 m of a tape measure, walls within 0.05 m, and scan to walkable in 90 s. RoomPlan's wall capture needs a second capture session, which crashed the scan on the phone; it is off by default.

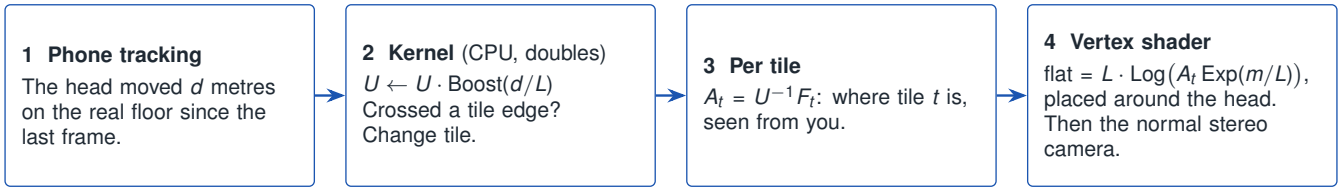
If asked

- **Which sensors?** LiDAR (mesh and depth), the rear camera and the IMU, through ARKit world tracking. No external tracker, no markers.
- **How accurate is the room?** A 5 cm grid, with free floor kept one cell short of every wall. The targets above are not measured yet.
- **Why not RoomPlan for everything?** AR Foundation exposes it only as furniture boxes. Running our own session for walls beside Unity's crashed the scan.

3 The maths: walking in curved space

The whole idea in three lines.

1. We never map the room onto curved space. **We map each step you take.**
2. Every frame runs two maps. **Step in:** the head's movement on the real floor becomes a movement in curved space (the exponential map, as a matrix). **Draw out:** every point of the curved world is laid flat around your head, at its true distance and true direction (the logarithm map).
3. Steps are added up, so where you are depends on the path you walked, not on where you stand. That one fact is why five rooms can meet at a corner.



Where a point lives

One number picks the geometry: $\kappa = -1$ hyperbolic, 0 flat, +1 spherical. Only the floor is curved; height stays ordinary ($\mathbb{H}^2 \times \mathbb{R}$ and $\mathbb{S}^2 \times \mathbb{R}$). A point of the hyperbolic floor is a 3-vector $p = (x, z, w)$ on the **hyperboloid**

$$x^2 + z^2 - w^2 = -1, \quad w > 0, \qquad \text{start point } o = (0, 0, 1), \qquad \langle a, b \rangle = a_x b_x + a_z b_z - a_w b_w.$$

The sphere is the same with the sign flipped: $x^2 + z^2 + w^2 = 1$. We use this model because every motion of the floor (a walk, a turn) is then a 3×3 matrix, and one motion after another is a matrix product, exactly as rotations work for a sphere. The Poincaré disc is used only to draw the map.

Lengths inside the kernel are in units of L, the curvature radius (curvature $K = -1/L^2$); multiplying by L gives metres.

	Hyperbolic ($\kappa = -1$)	**Sphere** ($\kappa = +1$)
Distance between p and q	$\cosh d = -\langle p, q \rangle$	$\cos d = p \cdot q$
Exp: flat vector t, $r = \lvert t \rvert$, to a point	$(\sinh r \cdot t/r,\ \cosh r)$	$(\sin r \cdot t/r,\ \cos r)$
Log: point p to a flat vector	$\operatorname{asinh}(\lvert p_{xz} \rvert) \cdot p_{xz}/\lvert p_{xz} \rvert$	$\operatorname{acos}(p_w) \cdot p_{xz}/\lvert p_{xz} \rvert$
Undo a motion M	$M^{-1} = \eta M^{\mathsf{T}} \eta$, $\eta = \operatorname{diag}(1, 1, -1)$	$M^{-1} = M^{\mathsf{T}}$

Step in: the walker's pose

The pose is a pair (tile, U): which tile you are in, and a 3×3 matrix U that says where you are in it and which way you face. Each frame the phone gives the head's floor displacement d (metres), and

$$U \leftarrow U \cdot \text{Boost}(d/L), \qquad \text{Boost}(\Delta) = \exp \begin{pmatrix} 0 & 0 & \Delta_x \\ 0 & 0 & \Delta_z \\ -\kappa\Delta_x & -\kappa\Delta_z & 0 \end{pmatrix} = \begin{pmatrix} 1 + (c-1)u_x^2 & (c-1)u_x u_z & s\,u_x \\ (c-1)u_x u_z & 1 + (c-1)u_z^2 & s\,u_z \\ s\,u_x & s\,u_z & c \end{pmatrix}$$

with $u = \Delta/\lvert\Delta\rvert$, $c = \cosh\lvert\Delta\rvert$, $s = \sinh\lvert\Delta\rvert$ (written out for $\kappa = -1$; the sphere uses cos, sin and a minus sign on the last row; at $\kappa = 0$ it is a plain translation). This is a **Lorentz boost**, the matrix of special relativity, with the two floor axes as space and w as time.

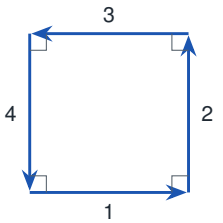
- A step larger than 0.2 m in one frame, or a frame where tracking is not Normal, counts as no step.
- U is put back onto the group every frame (Gram–Schmidt with respect to η), so rounding never builds up.
- The kernel is one class, `HolonomyWalkerKernel`: flat, hyperbolic {4,5} or {4,6}, or the sphere, by configuration.

Why the path matters (holonomy)

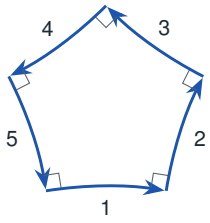
Take the instruction "walk s, turn left 90°": $g = \text{Boost}(s/L) \cdot \text{Rot}(90°)$. In each geometry g is a rotation by an angle ϕ about some fixed point, and the walk closes after n sides when $\phi = 360°/n$:

$$\cos\frac{\phi}{2} = \frac{\cosh(s/2L)}{\sqrt{2}} \ \text{(hyperbolic)}, \qquad \cos\frac{\phi}{2} = \frac{\cos(s/2R)}{\sqrt{2}} \ \text{(sphere)}.$$

Sides to close	**Geometry**	**Side length**
3	sphere	$s/R = \pi/2 = 1.5708$
4	flat	any
5	hyperbolic {4,5}	$s/L = 1.0613$
6	hyperbolic {4,6}	$s/L = 1.3170$
never	hyperbolic	$s/L \geq 1.7627$



Your floor (flat): closes after 4 sides



The hyperbolic world: the same walk closes after 5

What you feel. Your floor is flat, so four sides bring your body back to the same spot. In the world they do not: after one lap you are one side-length away and turned 90°, so you stand in the next room. Five laps bring the first room back. In general, carrying a direction round a closed loop turns it by (area of the loop)$/L^2$ (Gauss–Bonnet).

Loop-fit rule. We choose L so that the square that fits *your* room is the five-corner square. From the scan's free rectangle $W_f \times D_f$: $s = \text{clamp}(\min(W_f, D_f) - 0.4\,\text{m},\ 1.0,\ 3.2)$, then $L = s/1.0613$ (five rooms per corner), $L = s/1.3170$ (six) or $R = 2s/\pi$ (planet). Example: $s = 2.0$ m gives $L = 1.88$ m.

The world: tiles with exact addresses

The hyperbolic floor is cut into square tiles, five at every corner: the {4,5} tiling. In units of L: centre spacing 1.0613 (its cosh is the golden ratio φ), corner angle 72°, corner distance 0.8425, area $2\pi/5$. The spacing is the same 1.0613 as above: "rooms per corner" and "sides to close" are one number.

- **Room to spare.** Tiles at 0, 1, 2, … steps away: 1, 4, 12, 28, 64, 148, 340, 780. That is about $\times 2.3$ per ring; flat squares give 1, 4, 8, 12, 16.
- **Exact ids.** Stepping through edge k is the matrix $S_k = R^k T H$, with R a quarter turn, $H = \mathrm{diag}(-1,-1,1)$ and $T = \begin{pmatrix} \varphi & 0 & \sqrt{\varphi} \\ 0 & 1 & 0 \\ \sqrt{\varphi} & 0 & \varphi \end{pmatrix}$. Every entry is $u + v\varphi$ with whole numbers u, v, so a tile centre is $(\sqrt{\varphi}\,X, \sqrt{\varphi}\,Y, W)$ with $X, Y, W \in \mathbb{Z}[\varphi]$: **six integers**, with the check $\varphi(X^2 + Y^2) - W^2 = -1$. Because $\varphi^2 = \varphi + 1$, a neighbour step needs additions only.
- **What that buys.** "Which room am I in" has no rounding error, ever. Walk five rooms round a corner and the id is exactly the first one again. A tile's content is a hash of its id, so a room is the same each time you return and nothing is stored. 64-bit integers are exact out to 42 straight tile steps.
- **Staying accurate.** Hyperboloid coordinates grow like e^d. So when you cross edge i the kernel does $U \leftarrow G_i^{-1} U$ and moves to the neighbour: U only ever describes a place inside one tile and stays small.

Draw out: the warp

On the CPU, for each drawn tile t: $A_t = U^{-1} F_t$ (in doubles, cast to float last). In the vertex shader (`AtlasWarp.hlsl`), for each vertex m, given in metres in its tile's own flat chart:

$$p = A_t \cdot \mathrm{Exp}(m_{xz}/L), \qquad t = L \cdot \mathrm{Log}(p), \qquad \text{world position} = (\mathrm{head}_x + t_x,\ \ m_y,\ \ \mathrm{head}_z + t_z).$$

- **Exact:** the distance and the direction of every point from your head. It is the map projection called azimuthal equidistant, centred on you. Rays from the head are straight in the picture, so what you see is what an eye in $\mathbb{H}^2 \times \mathbb{R}$ would see.
- **Shrunk:** sideways lengths at distance d are drawn at scale $(d/L)/\sinh(d/L)$: 0.85 at one unit, 0.30 at three. That is correct, not an error. Things fall away faster than in a flat room, which is how so many rooms fit in view.
- **Stereo:** the warp is centred on the head, not on each eye; the two eye cameras then give correct depth.
- **Fog for free:** flat distance from the head divided by L is the true geodesic distance. Fog starts at $\rho - 1$ and is solid at $\rho = 3$ units. A tile is drawn when its centre is within $\rho + 0.8425$: at most 71 tiles.
- **Seamless tiles:** each tile mesh is a grid in the Klein model, where straight lines are geodesics, so neighbours share their edge exactly. Grid point k goes to the chart point $L \cdot \mathrm{atanh}(|k|) \cdot k/|k|$.
- **Your room:** the scanned room and its furniture sit in tile zero and pass through the same warp. At $\kappa = 0$ the warp is the identity, so the room matches the real one. Curvature is ramped in over 4 s, and only while you stand still (head speed at most 0.1 m/s).

If asked

- **Why the hyperboloid and not the Poincaré disc?** Motions are plain matrices and composing them is a matrix product, as with rotations of a sphere; it is also numerically stable. The disc is only our map picture: $\rho = \tanh(r/2)$.
- **What is distorted?** Nothing along a ray from your head. Sideways lengths shrink with distance, exactly as hyperbolic space looks from inside.
- **How do you avoid drift?** Integer tile ids, a pose kept inside the current tile, re-orthonormalising every frame, doubles on the CPU and one cast to float at the end.
- **How do you know it is right?** Unit tests on Exp and Log, the holonomy, the closing of the five-sided lap and the tile ids; a recorded five-sided lap closes to under 0.25 m in the end-to-end test. The pass mark for the shader is 1 mm from the CPU reference out to three units.

4 Rendering and performance

One frame, in order



Steps 1 to 7 never wait for the network, and Bump Guard runs before anything that could fail. A space is a module (`ISpaceModule`: configure, enter, tick, exit) hosted by one scene, `SpaceLab`, with a 0.25 s fade between spaces. Everything is built in code at boot.

Stereo and the lens

- Two eye cameras 64 mm apart, each drawing into its own texture: 1118×1032 px with $4\times$ MSAA. No XR plug-in.
- One full-screen pass then bends both pictures for the lenses: $r' = r\,(1 + K_1 r^2 + K_2 r^4)$. The lens numbers are Cardboard-class placeholders, tunable live from the Console.

- Optional gyro prediction (off by default) turns the eyes up to 40 ms ahead. The kernel and Bump Guard never see it.

One shader, a diagram look

Every surface uses `Atlas/Surface`: one pass, one variant, no keywords.

- Colour from a small palette, shade baked into vertex colours, **no lighting from normals**; grid and edge lines from screen-space derivatives; fog to the horizon colour. The warp is its vertex stage; the portal clip and the sight-line gates are in its fragment stage.
- No lights, shadows, HDR or post-processing. One material layout and pooled materials, so the SRP Batcher can batch everything.
- Why flat: lightmaps and normals cannot follow warped geometry, and a line drawing of a curved world reads better than a lit one.

Budgets and the thermal ladder

Targets per frame: at most 300 draw calls and 500,000 triangles for both eyes. Curved Walk is the heaviest space (240 draw calls, 370,000 triangles); Home is 80 and 250,000.

Tier	Render scale	MSAA	Draw radius ρ	Glued copies	Furniture	Frames per second
Nominal	0.8	4×	3	5×5	full	60
Fair	0.7	4×	3	5×5	light	60
Serious	0.6	2×	2	3×3	light	60
Critical	0.6	off	2	3×3	light	30

The ladder steps down one tier at a time, at most once per 20 s, when the phone's thermal state rises or the GPU time's 95th percentile passes 9 ms, then 12 ms. Quality drops before the frame rate does; the floor is 30.

Status. No frame rate, frame time or thermal figure has been measured on a phone yet; the target is 60 frames a second. Measured elsewhere: 0 bytes of managed allocation over 100 frames in S0 to S6 (headless), and a tile-crossing frame's kernel and upload at 0.94 ms at the 95th percentile (Editor). Motion-to-photon is estimated at 55 to 95 ms, not measured. The app carries its own performance tour to produce the real numbers.

If asked

- **Does it hold 30 frames a second?** It is built for 60, with a ladder that gives up quality before frame rate and a floor of 30. Say the number only once it is measured on the phone.
- **How is stereo made?** Two cameras into two textures, then one lens pass. We wrote the rig ourselves.
- **Why does it look like a diagram?** By choice: one cheap shader that still works when the geometry is warped.

5 The other spaces

S1 Glued: torus, Klein bottle, 3-torus

- **Maths.** A torus is the flat floor with points identified under two translations A and B. In the Klein bottle B also mirrors left and right. The 3-torus adds a vertical translation C of 2.6 m.
- **Built as a lattice of copies,** fixed to the floor: copy (col, row, layer) is $A^{col} B^{row} C^{layer}$, a mirror flag plus a translation. We draw 5×5 copies of the cell (3, 5 or 7 by thermal tier; 27 for the 3-torus). In the Klein bottle the odd rows are drawn mirrored.
- **Nobody is teleported.** The walker is never moved; a copy of the walker, with their hands, stands in every other cell. So you see your own back, and in the Klein bottle your copy waves the other hand.

S3 Aligned Cells: the hyperbolic world from flat pieces

- The real floor is cut into flat square cells of side g (about 1.5 m). Each edge you cross is one step in the same {4,5} tile graph as Curved Walk. Four cells meet at a post on the floor, but five tiles meet there in the world, so one walk round a post ends in a fifth room.
- **All the curvature sits at the posts.** The angle round a post is $5 \times 90° = 450°$, a curvature of $-\pi/2$ per post, and $4 \times (-\pi/2)/5 = -2\pi/5$ per tile: the same as the smooth tile. The equivalent radius is $0.892\,g$.
- **Sight-line gates.** What a cell shows depends on which way round the posts you look at it. The shader clips each pixel by the sign of (post − eye) × (pixel − eye), with up to four gates, per eye, and no stencil. At most 57 cell instances are drawn.

- Every piece is flat, so the picture matches the real room and the furniture stays where it really is. It is the fallback way to walk the hyperbolic world.

S4 Small Planet: the sphere

- The {4,3} tiling: the six faces of a cube blown onto a sphere, all always drawn. The same kernel with $\kappa = +1$: $U \leftarrow U \cdot \mathrm{Boost}(d/R)$ with cos and sin.
- $R = \max(2s/\pi,\ 2.0\,\mathrm{m})$, so the planet is at least 12.6 m round. Three right-angle turns close a triangle when a side is a quarter of the way round: 3.14 m at the default $R = 2.0$ m. In a smaller room, lower `planet.R` so that the lap square closes it.
- Fog runs from 0.5π to 0.8π radians, and pixels beyond 2.6 radians are discarded, because the log map smears near the point opposite you.

S5 Impossible Rooms: portals

- The rooms overlap on the same real floor, so every portal's transform is the identity. **Nobody is teleported**, and the real floor stays under your feet. A 1 m booth holds a hall as large as the whole floor; a corridor with four right turns ends in a different room.
- **Which room you see is chosen per pixel.** Each visible doorway takes a stencil value (15 at most). A mask marks the doorway and resets depth; the room behind is drawn with the stencil test and a clip plane. There is no extra camera and no render texture, one level deep.
- **Crossing.** When the head passes the doorway's plane, the current room swaps. The layout is generated from the free rectangle (doors 0.9 m wide, 2 to 6 rooms).

S6 Slicer: four dimensions

- A 4D body is a mesh of tetrahedra with 4D vertices: the tesseract (48 tetrahedra), the 5-cell, 16-cell and 24-cell, a ball, and tubes of 18,000.
- **The cut.** Every frame, on the CPU in a Burst job, each tetrahedron is cut by the hyperplane $n \cdot x = w_0$. With $f_i = n \cdot x_i - w_0$ at each vertex, every edge whose ends have opposite signs gives the point $x_i + \frac{f_i}{f_i - f_j}(x_j - x_i)$. Three such points make a triangle, four a quadrilateral.
- **Controls.** Walking along a lane on the floor moves the cut along w. The body turns in the XW, YW and ZW planes. Beside the slice is the shadow, $xyz \cdot f/(f - w)$ with $f = 2$.
- **Check.** The unit tesseract cut by $x + y + z + w = c$ is a regular tetrahedron of area 0.8660 at $c = 0.5$ and a regular octahedron of area 6.9282 at $c = 2$.

If asked

- **Do the portals teleport you?** No. All the rooms share one floor; a stencil mask picks the room for each pixel.
- **Torus or Klein bottle: what changes?** The same lattice. The Klein bottle mirrors every other row, so your copy waves the other hand.
- **Where is the curvature in Aligned Cells?** At the posts: 450° round each one, which adds up to the same $-2\pi/5$ per tile as the smooth world.
- **Is the 4D real?** Yes. Real 4D tetrahedra are cut by a hyperplane every frame, checked against known slice areas.

6 Safety: Bump Guard

The headset view is generated, and in a curved space the virtual walls are not where the real walls are. So safety cannot come from the picture. **Bump Guard reads only the physical layer**: the real head pose, the scan's distance field and the live LiDAR depth image. No space feeds it anything, and it runs first in every frame.

Every frame

1. **New obstacles from live depth.** The depth image is sampled. A return that is 0.10 to 2.0 m above the floor and more than 0.30 m nearer than the scan predicts marks a 5 cm cell: a person who walked in, a moved chair. The cell clears when three images see through it, or after 30 s.
2. **Velocity.** The head's movement over the last 0.2 s.
3. **Clearance.** clear(p) is the smaller of the scan's distance field and the distance to the nearest marked cell. *now* is the clearance at the head; *path*(T) is the smallest clearance on the straight line of the next T seconds.
4. **Two levels.** WARN when now $<$ 0.30 m or path(0.8 s) $<$ 0.05 m. STRONG when now $<$ 0.20 m or path(0.4 s) $<$ 0.05 m.
5. **Hysteresis.** A level rises at once and falls only after the thresholds plus 0.05 m have held for 0.3 s, so the cue does not flicker.

6. **Tracking lost.** It falls back to depth alone (the nearest return in seven 10° sectors) and says “stand still” after 0.5 s.

- **The cue** is fixed to the head and never warped. WARN: an amber chevron on the hazard’s side, a tick, two haptic pulses a second. STRONG: a larger red chevron, a tone, six pulses a second. There is no off switch.
- **Independence.** The safety assembly may not reference the link or the telemetry; a test enforces it. A Wi-Fi drop cannot touch it.
- **Limits.** Live depth covers only where the head looks (0.3 to 5 m), and nothing lower than 0.10 m is covered.

Status. Proven headless, on synthetic depth and recorded walks: 20 scripted approaches warn at least 0.6 s before contact; ten wall approaches at 0.5 to 1.0 m/s lead by 0.85 to 0.93 s; a recorded lap gives no STRONG cue and two corner WARNs. The per-space run still skips the three curved spaces (S2 to S4). The on-phone checks (20 of 20 real approaches, with a spotter) are not recorded yet. The cost target is 0.3 ms a frame.

If asked

- **How do you stop a walker hitting a wall?** We predict the head’s path 0.4 and 0.8 s ahead against the real obstacles, and a chevron, a sound and a haptic pulse point at the hazard.
- **Virtual walls are not real walls. Why does it work?** Bump Guard ignores the virtual world. It uses the real pose and the real scan only.
- **What about a person who walks in?** Live depth marks anything more than 0.30 m nearer than the scan predicts, where the head is looking.

7 Tracked hands and input

The walker cannot see their own hands in a generated view. Tracked hands draw each hand where it is felt, and let a fingertip press and a pinch select. Gaze dwell (0.7 s), the headset lever and the gamepad always work as well.

From a camera frame to a hand in the room

1. **Detect.** Apple Vision’s hand-pose request (`VNDetectHumanHandPoseRequest`) runs in our Swift plugin on the camera frame ARKit already has: 21 joints per hand as picture points with a confidence each, up to two hands, 30 times a second. No image is stored.
2. **Depth.** LiDAR depth is sampled at every joint (the median of 3×3 pixels) and at up to 64 points across the palm.
3. **Palm.** A plane is fitted to the palm samples by least squares (outliers over 2.5 cm dropped). It is valid from 0.32 to 0.90 m; nearer than that, the distance comes from the hand’s known size.
4. **Fingers, bone by bone.** Each joint lies on its camera ray at one bone’s length from its parent, which gives two candidates. The eight chains of a finger are scored by anatomy, by LiDAR and by the last frame. LiDAR never places a fingertip; it only chooses.
5. **Hand size.** Bone lengths and scale (0.80 to 1.25) are learnt per walker from 30 square-on sightings.
6. **Frames.** Pixel, to the camera at capture time, to the ARKit session, to the room. A head turn after the capture therefore does not move the hand.
7. **Smooth and predict.** A One-Euro filter (palm 1.5 Hz, fingers 2.5 Hz); the palm is predicted up to 50 ms ahead; a result older than 0.2 s is dropped. A lost hand holds for 0.25 s, then fades.
8. **Draw.** A 790-triangle hand mesh, skinned on the CPU, that fades at the edge of the camera’s view.

Gestures

- **Pinch selects what you look at.** The measure is the thumb-tip to index-tip gap divided by the palm width. It closes under 0.25 on three detections in a row and opens over 0.40. It is ignored in a hand’s first 0.3 s, above 1 m/s, and within 0.4 s of the last pinch. It fires the target under the gaze at once, with no dwell.
- **Fingertip press.** The index fingertip hovers within 8 cm of a target’s face, arms 2.5 cm in front of it, and presses when it crosses a plane 1 cm behind the face while moving more in than across. In a hands session the targets sit 0.48 m from the camera and 25° below the horizon, so looking at them brings the hands into view.
- **Touch ring.** Reaching for a scanned wall or table shows a ring under the fingertip from 8 cm; at 1.5 cm it is contact, with a tap sound. The real surface is there to be felt.

With the rest of the app

- **Bump Guard.** A box round each hand is masked out of live depth, only at the hand’s own depth and never beyond 0.80 m. The scanned map is never masked. While a warning shows, the hands fade and fire nothing.
- **Heat.** 30 Hz normally, 15 Hz when the phone is warm, stopped when it is hot; 5 Hz when no hand has been seen for 20 s.

- **Field of view.** The camera sees about 72° × 44°; each eye sees about 100° × 96°. Hands are tracked only inside the camera's view.
- **Glued rooms.** Every copy of the walker carries the walker's hands, in the same mesh, with no extra draw call.
- **Demo tip.** First hold an open hand square to the camera for a second or two. Until the hand's size is learnt, a press nearer than 0.32 m may not fire.

Status. Built and merged; 322 hands tests, 117 input tests and 12 end-to-end tests pass headless. Not yet run in the Editor or on a phone, and `hands.enabled` is off by default. Hands sessions stay standing until the walking check with a spotter has passed. Fingertip accuracy and latency are not measured: the expected result age is 70 to 110 ms, and a headless test with synthetic noise holds the fingertip within 30 mm at the 95th percentile.

If asked

- **Why Apple Vision and not MediaPipe?** Vision reads ARKit's own frame, so it never fights the AR session for the camera, and it needs no package or model file. MediaPipe is the named fallback behind the same interface; it is not built.
- **Vision gives 2D joints. Where does 3D come from?** LiDAR depth for the palm, then bone lengths along each joint's camera ray for the fingers.
- **Do raised hands trip the collision warning?** They are masked from live depth at the hand's own depth. The cost we accept: a hand held ahead hides what is right behind it.

8 The Forge: furniture from photos, with TripoSG

Every scanned object gets a real-looking mesh, generated on the Station from its photos. Until it arrives, the phone shows a simple first shape.

How TripoSG works

The model is from the paper "TripoSG: High-Fidelity 3D Shape Synthesis using Large-Scale Rectified Flow Models" (arXiv 2502.06608, February 2025; VAST, The Chinese University of Hong Kong, The University of Texas at Austin and Shanghai AI Laboratory).

- **A shape is a signed distance field (SDF):** for any point, the distance to the surface, negative inside. The surface is where it is zero. This holds finer geometry than an inside/outside grid.
- **A VAE compresses the shape.** A transformer packs 20,480 surface points with their normals into latent tokens (up to 2,048 tokens of 64 channels), trained with SDF, surface-normal and eikonal losses.
- **The generator is a rectified-flow transformer** of about 1.5 billion parameters (the public weights). Rectified flow learns the velocity along a straight path from noise to a shape's latent; generating is integrating that path in N steps (we use 50).
- **The photo steers it** through cross-attention in every block, on DINOv2 image features of a 224 × 224 input, with classifier-free guidance.
- **A mesh comes out** by decoding the SDF on a grid (512 cells a side) and extracting its zero surface (marching cubes; the package `diso` does this on the GPU).
- **Training:** 2 million curated objects. **Output:** shape only, with no colour and no real-world size. Our clean-up adds both.

Our pipeline

1. **Capture** (phone, during the scan). Up to 96 key frames are kept. Each object gets 1 to 3 views at least 30° apart, taken from 0.6 to 3.0 m. A crop is the object's projected box plus 12 % a side, squared, at most 768 px.
2. **Upload.** The crops, with the box size, yaw and category, the cameras, and a 256-byte height profile of the object from the LiDAR scan.
3. **Matte.** The background is removed (`rembg`); the object is set on white, square, with a 10 % margin.
4. **Generate.** TripoSG: 50 steps, guidance 7.0, fixed seed, half precision, 90 s time-out.
5. **Clean-up.** Weld and decimate; turn the mesh to the pose that best fits the scanned profile; scale it to the scanned box; regrid so no edge is longer than 0.25 m (the warp bends vertices, so long edges would stay straight); colour each face from the photos; bake shade and ambient occlusion into vertex colours; make a light copy.
6. **Validate.** One mesh, no texture, 1,500 to 15,000 triangles, at most 2 MB, within 10 % of the scanned box. The room's budget is 32 meshes and 120,000 triangles, shared by surface area.
7. **Deliver.** A GLB file named by its SHA-256. The phone hears of it through the Hub (or polls every 2 s), checks the hash and the contract off the main thread, and fades the mesh in over 0.3 s.

- **First shapes.** At once, with or without the Forge: a template fitted to the scan (chair, couch, bed), a shape cut from the scan, or a box; at most 1,500 triangles. It stays if the Forge is away or a job fails.

- **Self-check.** Each result is scored against the LiDAR profile. A poor fit tries the next photo, at most three generations per object, and the best one is published.

Status. TripoSG has run on an RTX 4060 laptop (4 October): 87 s per object at 50 steps, against a 60 s target. No run is recorded on the Station's RTX 5070. A commit of 4 October notes three phone scans that sent the Forge no photos, so check the phone-to-Forge path end to end before calling it live. The tests use a procedural stand-in for the generator.

If asked

- **Why TripoSG?** MIT licence, one photo in, and it fits an 8 GB graphics card. TRELLIS needs 16 GB and SAM 3D Objects 32 GB.
- **Why not on the phone?** 1.5 billion parameters, about 8 GB of weights and a CUDA-only decoder. The phone shows first shapes and downloads a file of at most 2 MB.
- **What if the mesh is wrong?** It is scored against the LiDAR profile and must pass the contract; if it fails, the first shape stays.
- **Licences?** TripoSG's code and weights are MIT. The `diso` decoder is CC BY-NC 4.0: fine for a demo, open for commercial use.

9 The Station and the link

The Station is the PC beside the walker. Four programs run on it.

Program	Built with	What it does
Hub	.NET 8 console; UDP 9050, HTTP 8080	Ends the phone link, relays the phone's state to Spectators, sends parameters and operator commands to the phone, writes the session logs, serves the Console and the pairing page.
Console	a web page served by the Hub	Live state at 10 Hz, a slider for every registered parameter, presets, and the operator commands: home, recentre, quality tier, STOP cue, demo lock, change space.
Spectator	Unity build for Windows	Redraws the walker's view from the relayed state with the same kernel and the same warp, beside the "flat twin" map (floor plan, or Poincaré disc in the curved spaces).
Forge	Python 3.11, FastAPI, port 8000	Turns object photos into furniture meshes (section 8).

- **Transport.** LiteNetLib over UDP, on the Station's own router with no internet. Pairing is a QR code holding `atlas://<station-ip>:9050`; the phone camera opens it and the app connects.
- **State.** One 202-byte message, 60 times a second. The Hub relays it unchanged and appends it to `state.bin`, so any walk can be replayed.
- **One source for the bytes.** The Hub compiles the app's own link source files, so the phone and the Hub cannot disagree about a byte of the protocol.
- **Clock and latency.** The Hub pings every client (eight times 50 ms apart, then four times a second), keeps a clock offset per client, and reports link latency at p50, p95 and p99.
- **Watchdog.** A client is *degraded* after 500 ms of silence and *offline* after 3 s. The Hub keeps the last value of every parameter and gives them back to a phone that reconnects.
- **Live tuning.** A slider on the Console becomes a `ParamSet`; the phone clamps the value and answers with what it applied. A demo lock makes the phone ignore changes.
- **Logs per session.** `state.bin`, `events.jsonl`, `perf.csv` (34 columns, one row a second), `phone.log`, `hub.log`. Motion logs are treated as health data and kept on an encrypted volume.
- **ReplayPhone.** A small program that plays a recorded or synthetic walk into the Hub, so the Console and the Spectator can be developed and tested with no phone.
- **No video is sent.** The state carries the head pose, the space, the tile id (six 64-bit integers), the walker matrix (nine doubles) and a few floats. The Spectator redraws the view itself and never integrates motion on its own.
- **Channels.** Control messages are reliable and ordered; the state is sequenced (a late one is dropped, never replayed). Clock sync is a four-timestamp exchange that keeps the sample with the smallest round trip; the target is 2 ms at the 95th percentile.
- **If Wi-Fi drops,** the walker notices nothing: the phone renders and decides everything, retries the link, and keeps writing its own telemetry.

Status. The Hub's 42 tests and the Console's 59 pass, and a rehearsal runs the Hub, the Forge and ReplayPhone together. No run of the Spectator's Windows build is recorded, and clock sync is not measured over Wi-Fi.

If asked

- **What does the Station do that the phone cannot?** It runs TripoSG on a GPU, shows the audience the walk, and lets an operator tune and stop it.
- **Why not stream video to the PC?** The state is 202 bytes; a stereo video stream would cost the phone heat and latency. The Spectator has the same kernel, so it can redraw exactly.

10 Use cases

Network Atlas (S7): why the internet fits hyperbolic space

- **Model:** Krioukov et al., *Hyperbolic Geometry of Complex Networks* (2010). 200 nodes are scattered in a hyperbolic disc of radius $R = 10.5$, more densely towards the rim. Two nodes are linked when their hyperbolic distance x is under R: $\cosh x = \cosh r \cosh r' - \sinh r \sinh r' \cos \Delta\theta$. Out come a few big hubs near the centre and many small nodes at the rim, as in real networks.
- **Greedy routing:** hand the packet to the neighbour nearest the target. No routing table, no map.
- **Built:** a disc on the Console and the Spectator, and a floor map in the headset: a Poincaré disc up to 1.2 m in radius ($\rho = \tanh(r/2)$). Looking at a node re-centres the map on it with the Möbius move $z \mapsto (z - a)/(1 - \bar{a}z)$ over 0.6 s. A walk-through version is written and unit-tested but not wired in.
- **Measured,** offline, on the same 1,000 random pairs: routing by hyperbolic distance delivers 100 % (seed 7) and 99.7 % (seed 83); routing by distance in the flat picture delivers 71.8 % and 72.6 %.
- **Say:** "a network *generated* by this model". **Do not say:** "the real internet", "greedy always works" or "flat space cannot route".

Lung: a breathing chest as a 4D object

- **Data:** TCIA 4D-Lung, patient 101, CC BY 3.0 (Hugo et al. 2016): a CT scan in 10 breathing phases, with real contours of the lesion, lungs, heart, oesophagus, spinal cord and trachea. A Station pipeline turns it into a case package (254 tests).
- **Time is the fourth axis.** Phase k sits at $\tau = 1.5\,\text{m} \cdot k/10$ and wraps round: $\mathbb{R}^3 \times S^1$. A surface followed through the ten phases becomes a 4D tube (each triangle sweeps a prism, split into three tetrahedra: 18,000 in all), and the Slicer cuts it. A straight cut is one phase. The shadow is everywhere the tumour goes. A tilted cut reproduces the motion artefact of a moving scanner table, $\tan\theta = c/v$.
- **The airway as a tree in hyperbolic tiles.** One generation is a turn tile plus a straight tile, and a branch's address is a word of lefts and rights. A tree doubles with every generation; so does the hyperbolic floor (about $\times 2.3$ per ring), so every branch gets a full-size room. On a flat floor the branches would collide.
- **Two Compasses.** At some forks the straight-line compass points down the wrong branch (8.7 % of forks in our tree); the geodesic one does not.

Status. The lung's organs and lesion are real contours, but the airway tree is model-generated, not this patient's. The app holds only a synthetic placeholder case today; the real package and the 4D tube are not wired into the headset. It is an education and research visualisation, not for clinical use: claim no clinical benefit.

FieldFit

A teaching demo of visual-field loss: Home seen through a 10° tunnel, then a lens that squeezes 54° back into view, done with a 512×512 lookup per eye in the final pass. The parts exist and are tested; it is not wired into the app.

If asked

- **Is the lung real patient data?** The lesion and the organs are real, public contours. The airway is generated by a model, and the app's bundled case is synthetic.
- **Does greedy routing always work?** No. It is measured, not guaranteed: 100 % and 99.7 % here against about 72 % in the flat picture.

11 How we know it works

- **Logic lives in engine-free classes** (`BumpGuardCore`, `FrameLoopCore`, the kernel, the link), so it can be tested without Unity.
- **The headless harness** (`tools/headless/verify.py`) compiles every assembly against the real Unity libraries for the phone, the Station and the Editor, and runs the engine-free tests on .NET 8. It exists because only one Mac can build to the phones. It cannot check shaders, scenes or a device.

- **Counts,** last recorded runs: 1,995 headless tests and 2,114 in Unity's EditMode, none failing, with every shader compiled for Metal; then 42 for the Hub, 59 for the Console, 53 for the Forge. The hands and the Network Atlas came after and add several hundred more.
- **Recorded walks** are replayed through the real frame loop: a recorded five-corner lap leaves tile zero and closes to under 0.25 m.
- **Gates:** G0 contracts frozen, G1 first light on the phone, G2 the curved-walk spike, G3 spaces in the Editor, G4 spaces on the phone at budget, G5 scan, Home and Bump Guard, G6 the Station link, G7 exit. Every headless line is proven; the device lines are still open.

What is proven where

As recorded in the repository on 4 October. Correct it with what you have seen since.

Part	Headless tests	Unity Editor	On a phone
Kernel, tiling, warp	pass	pass, shaders compile	no curved walk recorded
Scan and room model	pass	pass	ran to the end; accuracy not measured
Bump Guard	pass; lead 0.85 to 0.93 s	pass	not recorded
Glued, Aligned Cells, Small Planet, Impossible Rooms, Slicer	pass	pass	not recorded
Tracked hands	pass (322 tests)	not run	not run; off by default
Forge and TripoSG	pass, with a stand-in generator		TripoSG ran on a laptop GPU (87 s); no photos arrived from three phone scans
Station link and Console	pass (42 and 59)		not recorded
Network Atlas	pass, one app test failing	not run	not run
Lung, FieldFit	the parts pass	the parts pass	not in the app
Frame rate			no figure yet

Before you say it to a judge.

- A frame rate: only once it is measured on the phone.
- "Any phone": it needs an iPhone with LiDAR.
- Hands, generated furniture and the lung in the headset: say "built and tested off the device" unless you have seen it live.
- TripoSG's authors: VAST, The Chinese University of Hong Kong, UT Austin and Shanghai AI Laboratory.
- The network result: "in our generated network, 100 % against about 72 %".